RELAZIONE PROGETTO RETI DI TELECOMUNICAZIONI

Introduzione

Il progetto si basa sulla realizzazione di un *Simulatore a eventi di un sistema a coda*. Il modello assegnatoci è quello di una coda M/M/c, ovvero una coda in cui:

- Gli arrivi sono determinati da un processo di Poisson di intensità **λ** e spostano il processo da uno stato i a i+1.
- I tempi di servizio per ciascun servitore seguono una distribuzione esponenziale di parametro **μ**.
- Il numero di servitori presenti nel sistema è **c**.
- Quando un utente termina il servizio e lascia il sistema, il processo passa dallo stato i allo stato i-1.
- Non c'è nessun limite al numero di utenti che possono essere contenuti nella coda FIFO.
- Vi è stabilità se **$\rho = \lambda/(c\mu) < 1$**.

Obiettivo

Il nostro obiettivo è quindi quello di creare una simulazione in Java, che ci permetta di ottenere gli elementi caratteristici del sistema a coda, ovvero:

- La distribuzione di **probabilità degli stati (Pk)**, cioè la frazione di tempo in cui nel sistema sono presenti esattamente k clienti.
- Il **numero medio di clienti nel sistema (Ls)** e il **numero medio di clienti nella coda (Lq)**.
- Il **tempo medio di permanenza nel sistema (Ws)** e il **tempo medio di permanenza nella coda (Wq)**.
- La **probabilità di attesa (**nel codice **probAttesa)**, ovvero la probabilità che un cliente trovi tutti i servitori occupati e debba attendere in coda.

Una volta ottenuti i parametri che ci interessano per verificare l'efficacia della simulazione verranno comparati con i dati teorici e costruiremo dei grafici con un programma scritto in python.

Metodo

Per costruire questo progetto abbiamo suddiviso lo schema generale di ciò che volevamo creare in singoli componenti, in modo da poterci suddividere il carico di lavoro e semplificare la realizzazione del tutto. Siamo partiti quindi con la definizione del modello teorico, dopo aver ripassato e raggruppato le formule necessarie siamo passati alla struttura del simulatore, e infine alla validazione dei dati ottenuti.

-Simulatore:
Il simulatore è a eventi discreti, il tempo infatti non scorre linearmente ma avanza da un evento all'altro, i due possibili eventi sono ARRIVO e PARTENZA e facciamo uso di una lista degli eventi futuri (una PriorityQueue ordinata per tempo) che restituisce sempre l'evento più vicino.

L'esecuzione scorre in questo ordine: prendiamo il file config.json, lo carichiamo all'interno della classe Config, che ci permette di validare tutti i dati e interrompere la simulazione nel caso in cui ci sia una configurazione non valida.
Definiamo inoltre due liste per le classi RisultatiRun e RisultatiScenario, questo perché andremo a effettuare n (indicate nel config come ripetizioni) simulazioni per ogni coppia di valori (**λ, c**) e **μ** fisso, in pratica:

- Viene scelto lo scenario, ovvero i valori λ, μ e c.
- Vengono effettuate n simulazioni con questi parametri e ogni simulazione viene salvata nella lista di RisultatiRun.
- Una volta effettuate tutte le n simulazioni per scenario, sono salvate nella lista dei RisultatiScenario.

Così al termine dell'esecuzione il programma avrà tutti i risultati delle simulazioni suddivisi per scenario e ripetizione.

Dopodichè i dati ottenuti vengono salvati all'interno di un file chiamato risultati.json. Questo file viene letto da un programma chiamato plot.py che tramite la libreria matplotlib costruisce i relativi grafici comparando i dati simulati con quelli teorici.

Risultati e osservazioni

Dai grafici ottenuti si può osservare che i valori simulati coincidono con quelli teorici: in tutti i grafici i punti della simulazione si trovano sulle curve teoriche, entro gli intervalli di confidenza al 95%.
Questo ci permette di affermare che **il simulatore è corretto**.

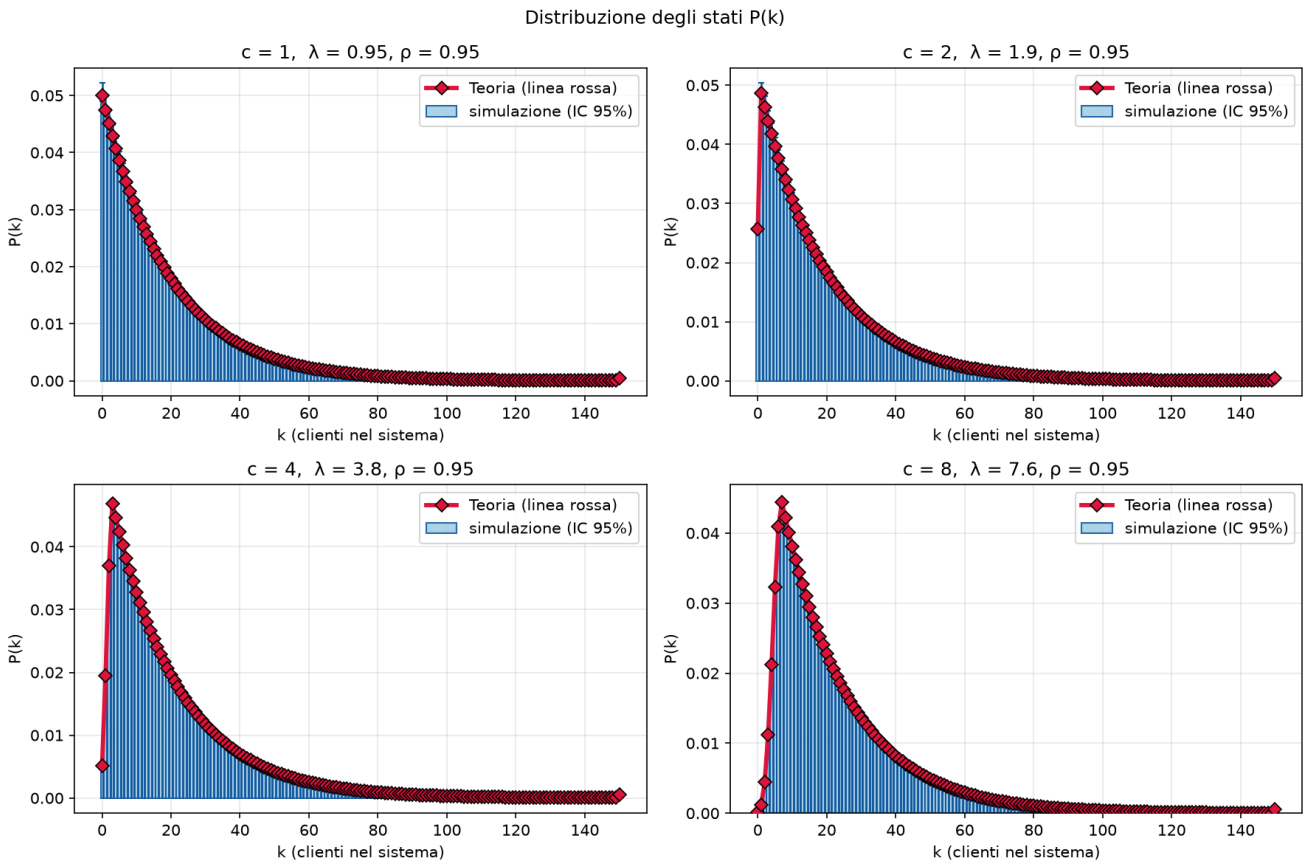
Le principali osservazioni sono le seguenti:

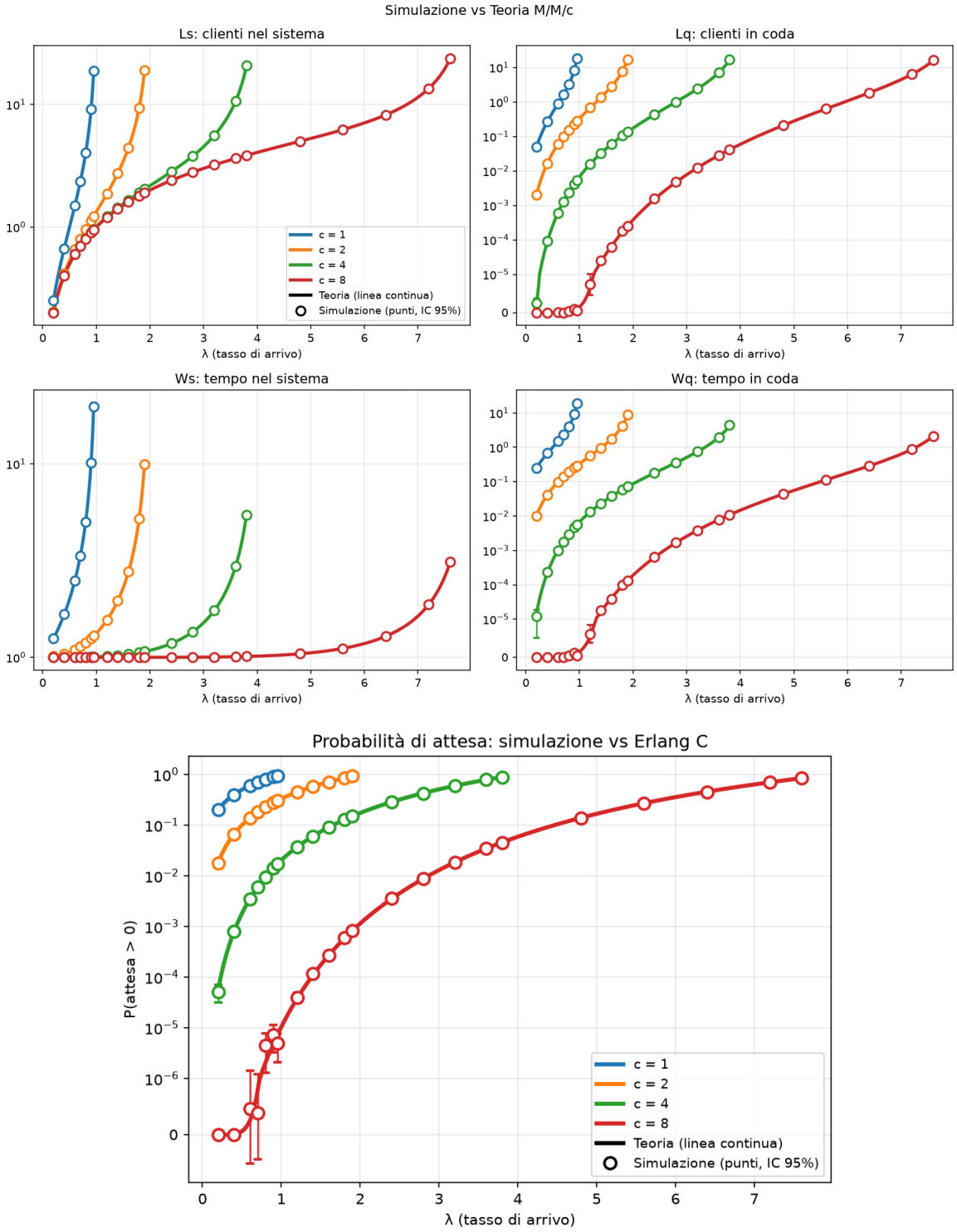
- Al crescere di λ, e quindi di $\rho = \lambda/(c\mu)$, le metriche Ls, Lq, Ws e Wq crescono lentamente a basso carico e poi in modo molto rapido quando ρ si avvicina a 1. Per questo motivo le curve si interrompono prima di $\lambda = c\mu$, dove il sistema non è più stabile.
- La probabilità di attesa tende a 1 quando ρ tende a 1. Nel caso c = 1 coincide con ρ, e infatti la curva corrisponde alla retta $y = \lambda$.
- A parità di ρ, un numero maggiore di servitori riduce molto i tempi di attesa. Per esempio, con ρ = 0.95 il tempo medio in coda Wq vale 19 per c = 1 e circa 2.1 per c = 8. Il tempo nel sistema Ws non può mai scendere sotto $1/\mu$, che è il tempo medio di servizio.

- Con molti servitori e un carico basso (per esempio $c = 8$ e $\lambda < 4$) la coda è quasi sempre vuota, quindi $W_s \approx 1/\mu$ e L_q e W_q sono molto vicini a zero.
- Nella distribuzione degli stati P_k ($\rho = 0.95$) per $c = 1$ il valore massimo è in $k = 0$ e P_k decresce in modo geometrico. Per $c > 1$ il massimo si sposta verso $k \approx c$, e poi la coda decresce con lo stesso andamento geometrico. L'ultimo punto del grafico ($k = 150$) rappresenta "150 o più clienti".
- L_q e W_q hanno lo stesso andamento, così come L_s e W_s, perché sono legati dalla legge di Little ($L = \lambda W$).

Alcuni punti presentano un errore più grande, e sono casi in cui non è un errore del simulatore ma il fatto che l'evento è molto raro:

- Per c grande e λ piccolo (per esempio $c = 8$, $\lambda \leq 0.8$) quasi nessun cliente deve aspettare, quindi nei grafici in scala logaritmica la simulazione dà 0 e gli intervalli di confidenza sono molto ampi. La teoria dà comunque valori piccolissimi ma non nulli.
- Per ρ alto (0.95) gli intervalli di confidenza sono più larghi, perché la coda cambia lentamente e servono simulazioni più lunghe per ottenere la stessa precisione.





In conclusione, la simulazione riproduce correttamente le prestazioni di un sistema M/M/c, e conferma che aumentare il numero di servitori migliora molto i tempi di attesa.